

分类号 TP302

学号

UDC

密级 公开

工学博士学位论文

基于先验模型与深度学习的图像去噪
方法研究

博士生姓名

学科专业 控制科学与工程

研究方向 多媒体信息系统与虚拟现实技术

指导教师

二〇二六年三月

Joint Demosaicing and Denoising of Bayer Images

Candidate:

Supervisor:

A dissertation

Submitted in partial fulfillment of the requirements

for the degree of Ph.D of Engineering

in Control Science and Engineering

March, 2026

目 录

表 目 录

图 目 录

摘 要

多模态大语言模型（MLLM）融合视觉、语言等模态的感知与生成能力，是当前人工智能研究的重要方向，其训练依赖分布式与并行计算。实际部署中的 GPU 集群往往由多代、多型号设备混合组成，存在显存、算力与通信的异构；而 MLLM 在结构上由编码器、投影层与 LLM 骨干等异构模块组成，训练时又广泛采用分阶段冻结策略，导致各阶段计算与显存需求差异显著。现有并行策略（如 Megatron-LM、ZeRO）多假设同构集群与单一模型形态，在异构环境下易产生负载不均衡、资源利用率低及冻结场景下的冗余计算与通信等问题。

本文从流水线划分与调度、显存优化两个层面开展研究。在划分与调度方面，提出 MMoH 框架：通过需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知的整数规划划分，在给定异构集群与 MLLM 配置下自动生成与当前训练阶段相匹配的流水线并行策略；通过提前终止调度器在连续冻结阶段省略不必要的反向计算与阶段间通信，在保持收敛性的前提下提升吞吐。在显存优化方面，针对现有框架（如 Megatron-LM）中激活重计算采用全局统一配置、在异构集群上难以同时兼顾显存安全与计算效率的问题，提出 HR-MMoH 的异构感知梯度检查点方法：系统梳理 Megatron 的重计算粒度与子模块配置参数，在 MMoH 划分基础上为各流水线阶段根据其显存上限与算力独立配置重计算粒度与子模块集合，显存紧张阶段采用更激进的检查点以降低 OOM 风险，显存充裕阶段采用较保守的配置以减少多余重算，并通过基于剖析的配置策略与 MMoH 形成“划分—调度—显存优化”的联合优化。在两种异构集群与两种规模 MLLM（3B、12B）上的实验表明，MMoH 在多种冻结场景下相对 Megatron-LM、Whale、Espresso 均可取得最高吞吐，最高可达约 1.77 倍加速，提前终止调度器可带来约 8%–19% 的额外增益；HR-MMoH 的按阶段差异化配置思路通过剖析实验得到验证，可为显存受限场景下的训练提供补充手段。本文为异构 GPU 集群上 MLLM 的高效并行训练提供了可参考的技术路径。

关键词：多模态大语言模型；异构 GPU 集群；流水线并行；梯度检查点；MMoH

Abstract

Multimodal large language models (MLLMs) integrate vision, language, and other modalities for perception and generation, and have become a major focus of artificial intelligence research; their training relies on distributed and parallel computing. In practice, GPU clusters are often composed of mixed generations and models of devices, exhibiting heterogeneity in memory, compute, and communication. Meanwhile, MLLMs are architecturally composed of heterogeneous modules such as encoders, projectors, and LLM backbones, and training commonly adopts stage-wise freezing, leading to significant differences in compute and memory demand across pipeline stages. Existing parallel strategies (e.g., Megatron-LM, ZeRO) mostly assume homogeneous clusters and uniform model structure, which in heterogeneous settings can cause load imbalance, low resource utilization, and redundant computation and communication under freezing.

This thesis addresses the problem from two aspects: pipeline partitioning and scheduling, and memory optimization. For partitioning and scheduling, we propose the MMoH framework: it uses demand-driven device topology identification, multi-granularity model abstraction, and freeze-aware integer-programming-based partitioning to automatically generate pipeline-parallel strategies that match the current training stage for a given heterogeneous cluster and MLLM configuration; an early-termination scheduler omits unnecessary backward computation and inter-stage communication at consecutive frozen stages, improving throughput while preserving convergence. For memory optimization, we address the fact that existing frameworks (e.g., Megatron-LM) use a single global activation recomputation configuration, which is ill-suited to heterogeneous clusters where memory-tight stages may still hit OOM while memory-rich stages incur unnecessary recomputation. We propose HR-MMoH with heterogeneous-aware gradient checkpointing: we systematically summarize Megatron’s recomputation granularity and submodule parameters, then assign each pipeline stage an independent recomputation configuration (granularity and submodule set) according to its memory headroom and compute, so that memory-tight stages use more aggressive checkpointing to reduce OOM risk and memory-rich stages use conservative settings to avoid redundant recomputation; a profiling-based configuration strategy is used and integrated with MMoH to form a joint “partitioning–scheduling–memory optimization” pipeline. Experiments on two heteroge-

neous clusters and two MLLM scales (3B and 12B parameters) show that MMoH achieves the highest throughput under various freezing scenarios compared with Megatron-LM, Whale, and Espresso, with speedups of up to about $1.77\times$, and the early-termination scheduler brings an additional 8%–19% gain; the per-stage heterogeneous checkpoint configuration in HR-MMoH is validated via profiling and provides a complementary means for memory-constrained training. This work provides a practical technical path for efficient parallel training of MLLMs on heterogeneous GPU clusters.

Key Words: Multimodal large language model; Heterogeneous GPU cluster; Pipeline parallelism; Gradient checkpointing; MMoH

符号使用说明

HPC	高性能计算 (High Performance Computing)
cluster	集群
Itanium	安腾
SMP	对称多处理
API	应用程序编程接口
PI	聚酰亚胺
MPI	聚酰亚胺模型化合物, N- 苯基邻苯酰亚胺
PBI	聚苯并咪唑
MPBI	聚苯并咪唑模型化合物, N- 苯基苯并咪唑
PY	聚吡咙
PMDA-BDA	均苯四酸二酐与联苯四胺合成的聚吡咙薄膜
ΔG	活化自由能 (Activation Free Energy)
χ	传输系数 (Transmission Coefficient)
E	能量
m	质量
c	光速
P	概率
T	时间
v	速度

第一章 绪论

1.1 研究背景

1.1.1 训练多模态大语言模型面临的挑战

近年来，人工智能技术发展迅速，应用场景不断拓展。深度神经网络（Deep Neural Network, DNN）在计算机视觉、自然语言处理、语音识别、工业检测、推荐系统等众多领域取得了突破性进展，已成为许多关键技术领域的核心基础 [?]。2017 年，Vaswani 等人提出基于自注意力机制（Self-Attention）的 Transformer 架构 [?]，摒弃了传统的循环与卷积结构，凭借并行化友好与长距离依赖的建模能力，迅速成为自然语言处理与跨模态建模的主流模型结构。

在自然语言处理（Natural Language Processing, NLP）方面，基于 Transformer 的预训练语言模型不断刷新各类基准。BERT [?] 通过双向编码与掩码预训练在理解类任务上取得显著优势；GPT-2 [?]、T5 [?] 等自回归或编码器 - 解码器模型则在生成与迁移学习上表现突出；GPT-3 [?] 与后续的 GPT-4 [?] 等大规模语言模型（Large Language Models, LLMs）更在少样本与零样本设定下展现出接近人类的泛化能力。在计算机视觉（Computer Vision）领域，Vision Transformer（ViT） [?] 及后续的规模化工作 [?] 表明：将图像切分为块（patch）并经 Transformer 编码，即可在 ImageNet 等基准上超越传统卷积网络。CLIP [?] 通过图文对比学习实现了开放词汇的视觉表征，DALL-E [?] 则展示了从文本到图像生成的可行性。上述进展共同表明，以 Transformer 为底座、以规模扩展为路径的技术路线已成为当前人工智能发展的主要范式之一。

多模态（Multimodality）指对两种或两种以上模态信息进行联合表示、推理与生成的技术范畴。在机器学习与人工智能中，常见模态包括文本、图像、视频、音频、表格以及各类传感器数据。多模态大语言模型（Multimodal Large Language Models, MLLMs）特指在参数规模巨大的语言模型基础上，融合对视觉、听觉等非文本模态的感知与生成能力的一类模型 [???]。这类模型既保留 LLMs 在语言理解、推理与生成上的优势，又能够直接处理图像、视频等多模态输入，实现“看图说话”“以文生图”等跨模态任务，与人类理想的人工智能模型相契合，被广泛视为迈向通用人工智能（AGI）的重要路径之一 [?]

从能力上看，MLLMs 可完成图像描述、视觉问答、图文对话、文档理解、以及根据文本生成图像或视频等任务，在智能助手、自动驾驶、内容创作、教育培训、工业质检等场景具有广阔应用前景。从结构上看，MLLMs 通常包含以下核心

组件（见图??）：（1）*Modality Encoder*，负责将图像、视频等非文本输入编码为特征表示，常采用预训练的视觉模型（如 CLIP、ViT）作为 *backbone*；（2）*Input Projector*，将编码后的多模态特征映射到与 LLM 词嵌入对齐的语义空间，常见形式包括线性层、Q-Former[?] 等轻量模块；（3）*LLM Backbone*，即大规模语言模型主体，承担主要的理解与生成计算，参数量通常占整体模型的 80%–90%；（4）*Output Projector*（在需要多模态生成时），将 LLM 输出映射到与目标模态生成器对齐的表示空间；（5）*Modality Generator*（在需要多模态生成时），根据投影表示生成图像、音频等目标模态，常采用扩散模型（如 Stable Diffusion）等。可见，LLMs 处于 MLLM 架构的核心，多模态能力是在其基础上的扩展与增强。

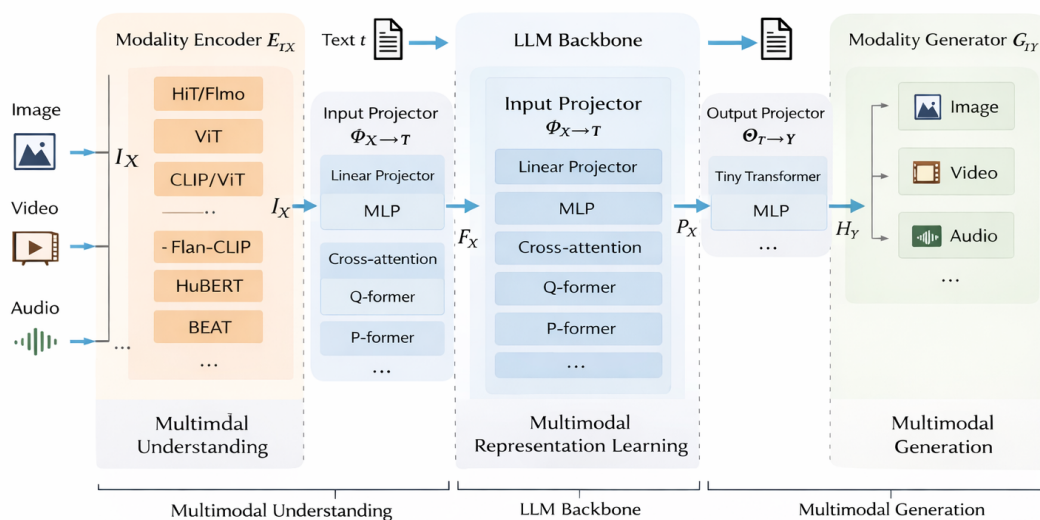


图 1.1 多模态大语言模型的核心组成部分

大量实证研究表明，模型性能随参数规模与训练数据规模的增加而持续提升，即存在所谓的“缩放律”（Scaling Laws）[?]。为追求更强性能，工业界与学术界不断推出参数量更大、数据规模更大的模型，从数亿参数（如 BERT）到千亿、万亿参数（如 GPT-3、DeepSeek-V3[?]、Qwen2.5[?] 等开源与闭源 LLMs），单卡或单机 GPU 的显存与算力已无法容纳整模型与完整训练流程[?]。相应的，多模态大模型的预训练与指令微调所需的数据量也急剧增长，从数百万图文对到数十亿级样本，单机算力、存储与 I/O 同样成为瓶颈。因此，在单计算设备上完成多模态大语言模型的训练已不现实，必须依赖分布式与并行训练，将模型参数、优化

器状态、激活值与数据分布到多台设备上，并通过通信协同完成前向与反向计算，才能完成这样的大规模的模型的训练 [??]。

目前，支撑大规模 MLLM 训练的主流并行策略可归纳为两大类。第一类是以计算与张量切分为核心的多维混合并行：数据并行 [??] 在各设备上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且在小模型上扩展性好；张量并行 [??]（如 Megatron-LM）将单层内的矩阵按行或列切分到多卡，降低单卡显存并保留层内并行，是千亿级稠密模型的主流选择；流水线并行 [??]（如 GPipe、PipeDream）将模型按层划分为若干阶段并分配到不同设备，通过微批次与调度重叠计算与通信，缓解单设备层数过多的问题；序列并行 [?] 与专家并行 [?] 则分别在长序列注意力与混合专家（MoE）结构中沿序列维或专家维切分，与张量、流水线并行组合后可进一步扩展模型规模。第二类是以显存优化为核心的零冗余优化器系列：ZeRO [??] 将优化器状态、梯度与参数分片存于各卡，前向与反向时按需聚合，在保持数据并行通信量的同时显著降低单卡显存；ZeRO-Infinity [?] 进一步将部分状态卸载至 CPU 与 NVMe，突破 GPU 显存墙；ZeRO++ [?] 通过量化与通信重算减少跨节点通信量，提升多节点训练效率。上述并行策略常与混合精度训练 [?]、激活检查点 [?] 等技术结合，共同构成当前大模型与 MLLM 训练的基础设施。然而，上述策略在设计时多假设同构集群与单一模型形态，在面对异构 GPU 集群与模块异构的 MLLM 时，仍存在效率与可扩展性方面的挑战，这也是本文关注的重点之一。

1.1.2 异构 GPU 集群的并行训练策略

多模态大语言模型的训练效率与成本高度依赖底层硬件平台。目前，工业界与学术界普遍采用 NVIDIA GPU 作为大模型与 MLLM 训练的主要加速器，并通过多卡、多机集群进行分布式训练。然而，NVIDIA 约每 1–2 年迭代一代计算架构并推出多款不同定位的 GPU，导致显存容量、算力与互联方式持续分化，实际部署中的集群往往由多代、多型号 GPU 混合组成，形成显存异构、算力异构与通信异构并存的局面 [??]。

图?? 概括了近年来 NVIDIA GPU 的架构演进。2016–2017 年的 Pascal（如 P100）与 Volta（V100）奠定了数据中心 GPU 与 Tensor Core 的基础；2018 年 Turing（RTX 20 系列）引入消费级光追与 INT8 推理；2020 年 Ampere 架构推出 A100（40GB/80GB HBM2e）与消费级 RTX 30 系列（如 RTX 3090 24GB），支持 PCIe 4.0 与 NVLink 3.0，成为当前许多实验室与云平台的主力 [?]。2022 年 Hopper 架构的 H100（80GB/96GB HBM3）针对 Transformer 与 LLM 训练做了专门优化 [?]，并广泛配备 NVLink 4.0 与 NVSwitch，单机多卡带宽可达数百 GB/s；同年 Ada

Lovelace 架构的 RTX 40 系列（如 RTX 4090 24GB）在消费级市场提供高性价比算力。2024 年起，Blackwell 架构的 B100/B200 等数据中心 GPU 逐步量产，采用 HBM3e、更高带宽的 NVLink5 与 PCIe 6.0，进一步拉大与旧代设备的性能与互联差距 [?]。按 NVIDIA 已披露的路线图，后续还将推出 Rubin（约 2026 年）、Rubin Ultra（约 2027 年），以及面向推理与能效深度优化的费曼（*Feynman*）架构（约 2028 年），后者将采用下一代 HBM 与 3D 堆叠内存等技术，进一步凸显代际间显存与互联的差异。与此同时，不同代际与型号在 PCIe 版本（3.0/4.0/5.0/6.0）、是否配备 NVLink/NVSwitch、以及 InfiniBand 与 RoCE 等节点间网络上的差异，使得通信带宽与延迟在集群内分布极不均匀。

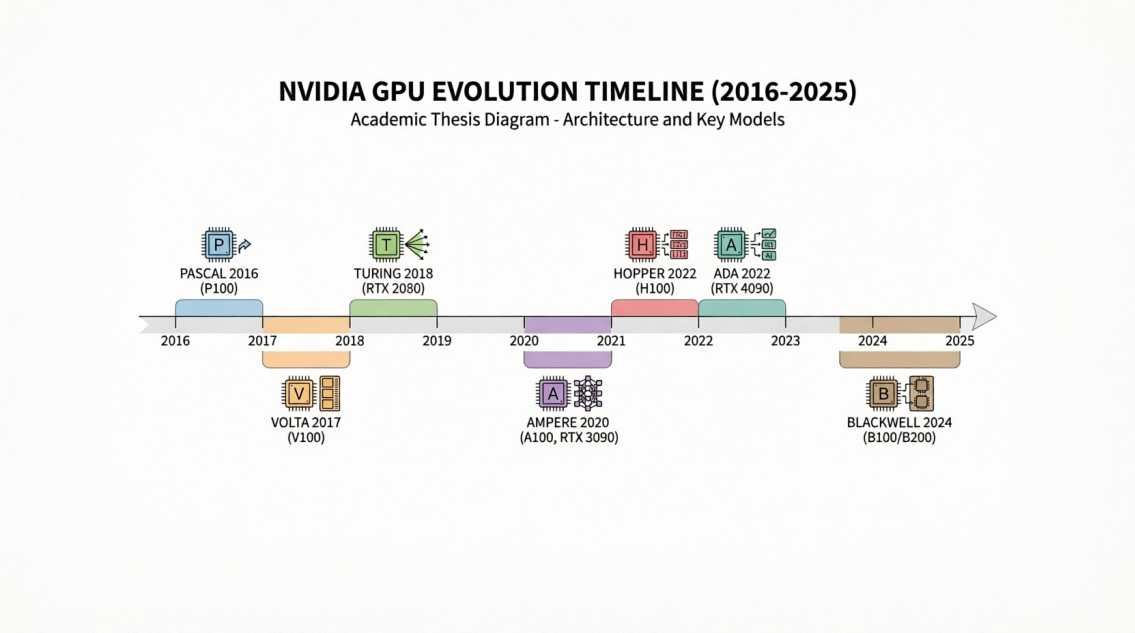


图 1.2 NVIDIA GPU 架构与代表性产品演进

在高校、研究所与中小企业的实际环境中，受预算与采购周期限制，集群常同时包含多代 GPU（例如部分 A100 或 H100 与大量 RTX 3090/4090、甚至更早的 V100/RTX 2080），单机内也可能混用不同显存与互联配置，难以做到“全集群同构”。而主流并行框架（如 Megatron-LM、DeepSpeed、PyTorch Distributed）在设计时多假设同构设备与统一拓扑 [?]，在异构集群上直接使用往往导致：负载无法按算力与显存合理分配、通信成为瓶颈或部分高端卡空闲、以及因“木桶效应”整体效率下降 [?]。因此，在显存、算力与通信三重异构的 GPU 集群上高效训练 MLLM，是许多机构共同面临且亟待解决的问题。

除了硬件设施固有的算力、显存和通信异构，模型异构也是现有并行训练框架无法高效训练多模态大语言模型的原因之一。负责将不同模态的输入数据编码

为模型可理解的 Modality Encoder 一般是视觉模型 (如 CLIP、ViT 等), 核心模块 LLMs 是常见的基座大语言模型 (如 GPT-3、Llama-3[?]), 具有庞大的参数量, 大约占据整个多模态大语言模型 80%–90% 的参数量。Modality Generator 一般是一个 Diffusion 生成模型 (如 Stable Diffusion[?]), 根据 LLMs 的输出产生图像、视频、音频等其它模态的数据。其中, LLMs 是由参数量很大的 Transformer 层堆叠而成, Encoder 可能由小的 Transformer 层或卷积层等其它神经网络组成, Generator 也可能是 U-Net[?] 等其他网络结构。前后两个 Projector 可能是 Transformer 层、Linear 层, 也可能是 Q-Former[?] 等其它网络结构。多模态大语言模型的这种网络结构和传统的大模型结构不同, 传统的大模型只有一种参数量固定的 Transformer 层, 每一层在特定 GPU 上运行所需的时间和显存都是固定的; 而对于多模态大语言模型, 模型的某一层变化多样, 在不同的 GPU 上运行的时间开销和显存开销都不同, 在异构 GPU 集群的环境下, 这种模型异构对于异构集群上训练效率带来的影响被进一步放大。

由于深度学习的飞速发展, 在计算机视觉、大语言模型、内容生成领域都涌现出很多性能强大的开源模型, 为如今多模态大语言模型的训练范式奠定了基础。Blip-2[?] 率先提出利用已经训练好的 SOTA 模型, 在训练中冻结部分模块, 参数更新仅仅发生在非冻结的模块, 模块之间使用一个小的神经网络连接起来进行模态转换, 取得了非常好的效果。自此以后, 基本上所有多模态大语言模型的训练都使用这种模式, 比如近年来有代表性的 NextGPT[?]、DreamLLM[?] 等多模态大语言模型工作。除了冻结, 多模态大语言模型的训练一般被分为多个阶段 (模态对齐、指令微调等), 不同的阶段需要冻结不同的模块, 侧重赋予模型不同的功能。由于一个模块被冻结后, 反向传播中的计算量会有所下降, 根据实验观察这种下降程度会随着冻结位置的变化和模块自身神经网络的特性而有所不同, 优化器也不再需要存储冻结的参数和可能的其余参数 (比如使用 Adam[?] 优化器需要存储的梯度的动量和方差), 这在很大程度上降低了模型训练所需要的显存。在异构 GPU 集群的环境下, 冻结带来的算力、显存和通信影响进一步影响了多模态大语言模型的训练效率。

本课题将结合现有的并行策略和实际 GPU 集群的异构场景, 基于 Megatron 等现有并行训练框架, 针对多模态大语言模型中模型异构、冻结带来的影响以及实际 GPU 集群中存在的存储、算力和网络的异构情况, 探索并实现一个高效的、细粒度的混合并行技术, 提供一个面向异构复杂环境下多模态大语言模型并行训练的自适应编译优化方案, 有效解决异构 GPU 集群下训练多模态大语言模型计算效率低、存储利用差、通信开销瓶颈等问题。

1.2 国内外研究现状

1.2.1 多模态大语言模型并行训练策略研究现状

纵观近几年的人工智能领域的发展不难发现，目前的基座模型、视觉领域模型、内容生成模型正在不断向着大规模、大数据的方向纵深发展，这直接导致了多模态大语言模型也越来越大。由于 **Scaling Law** 的本身特性，经过实验证据表明，提升大模型的性能表现的两条主要途径是增大模型参数和扩大数据规模 [?]。通过提升模型参数，大模型可以有效地增强模型对于复杂场景的拟合能力和泛化能力；通过扩大数据规模，大模型可以获得更多的样本去学习充足的特征表示，从而提高其特征提取和表征能力 [?]，更好地适应于目前复杂的应用场景需要。

自 **Transformer** 提出以来，模型规模迅猛增长。从 2018 年 **BERT** 的 3.4 亿参数量，到 2019 年 **GPT-2** 的 15 亿参数量，再到 2020 年 **GPT-3** 的 1750 亿参数量，2021 年阿里巴巴 **M6** 的 10 万亿参数量等。在单个硬件加速器上训练大语言模型已经不再现实，并行训练是解决模型规模过大问题的必要途径。除此之外，由于大语言模型目前主要是数据驱动的，为了能够获得满意的性能表现，训练的数据量也在不断扩大。例如图片数据集 **JFT-3B** 包含接近 30 亿张图片，**Common Crawl** 网页数据中包含数千亿 **tokens**，存储图像数据集 **Open Image**[?] 需要 18TB 的存储空间，视频分析的 **YouTube-8M**[?] 数据集甚至需要高达 1PB 的存储空间。在单个硬件加速器上已经无法存储大模型的完整数据集，并行训练同样是解决数据集规模过大问题的必要方式。综上所述，模型参数量与数据集规模已经远远超出了单个设备的存储上限，并行训练是训练多模态大语言模型的必由之路。而由于 **GPU** 产品的快速更新换代、网络连接的不断发展变化，实际的并行训练平台中往往包含着许多不同存储、不同算力、不同带宽网络的异构设备，这些 **GPU** 及其网络之间的异构性会严重影响多模态大语言模型的训练性能。因此，面向异构 **GPU** 环境设计多模态大模型并行训练方法，实现自感知、自适应、细粒度的编译优化技术，进而提高算力资源的利用率、充分利用存储资源以及解决通信开销的瓶颈，对于未来的多模态大语言模型的训练是十分必要的。

1.2.2 异构 GPU 集群并行训练策略研究现状

正如前文所述，并行训练是当前工业界与学术界训练多模态大语言模型的主要手段，主流做法可概括为两类：以计算与张量切分为核心的多维混合并行，以及以显存分片为核心的零冗余优化器（**ZeRO**）系列。

多维混合并行通过在不同维度上划分模型或数据以降低单机负载，包括数据并行、张量并行、流水线并行、序列并行、上下文并行与专家并行等 [?]。各方

式可组合使用，在保证正确性的前提下通过通信协同完成前向与反向计算。零冗余优化器 ZeRO[?] 在数据并行的基础上，将优化器状态、梯度与参数分片存于各卡，按需聚合，在通信量可控的前提下显著降低单卡显存，使千亿、万亿参数规模的训练成为可能。

NVIDIA 提出的 Megatron-LM[?] 是多维混合并行的代表性框架，通过联合使用张量并行、流水线并行与数据并行，已支撑在数千张 GPU 上训练千亿至万亿级参数的大语言模型。Megatron-LM 的后续工作持续针对大模型训练的显存与效率进行优化：Korthikanti 等在 MLSys 2023 上发表的论文 [?] 针对超大 Transformer 的激活显存与重计算开销，提出序列并行技术，与张量并行协同使用，弥补了单一张量并行无法在归一化层和 Dropout 层切分中间激活值的不足。

然而，Megatron-LM 在设计时以同构 GPU 集群与单一形态的 Transformer 模型为前提，未考虑设备异构与模型异构。在实际异构环境下使用 Megatron 训练多模态大语言模型时，往往只能将不同型号、不同显存与带宽的 GPU 当作同构设备统一配置，其典型并行策略如图??所示。对于 MLLM 中结构各异的 Modality Encoder、LLM Backbone、Projector 与 Modality Generator，Megatron 也缺乏针对性的细粒度划分与映射：编码器与生成器的并行方式通常被动跟随 LLM 主干的划分，难以根据各子模块的算力与显存需求、以及不同 GPU 的 capability 做自适应分配。其结果是，显存小、算力弱或通信慢的 GPU 成为瓶颈，高端 GPU 无法充分发挥，整体集群利用率与训练效率下降 [? ?]。

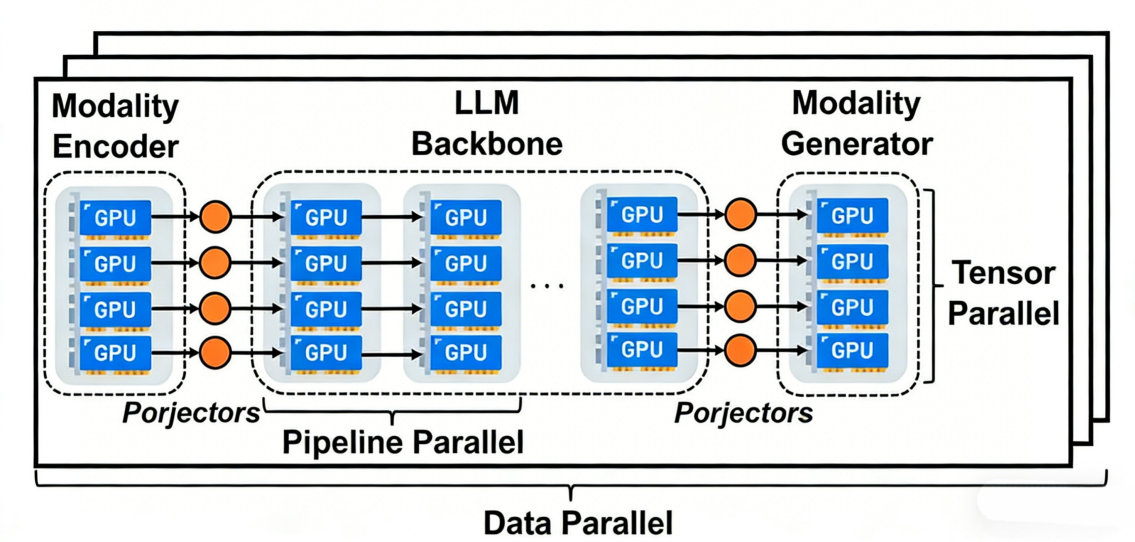


图 1.3 使用 Megatron 训练多模态大语言模型（同构假设下的典型配置）

ZeRO[?] 由微软 DeepSpeed 团队提出，通过优化器状态、梯度与参数的分片与按需聚合，在数据并行框架下降低显存并提升可扩展性。但在面向多模态大

语言模型的异构集群场景中，ZeRO 仍存在明显局限。一方面，ZeRO 的划分与通信模式与具体硬件的算力、显存及拓扑解耦，编译与调度阶段无法感知算力异构与显存异构，因而难以针对不同设备做负载与显存的均衡分配，易造成算力与存储的浪费。另一方面，在大规模分布式训练中，通信量与设备数量同步增长，而 ZeRO 未针对异构网络（如混合 NVLink、PCIe 与 InfiniBand）做拓扑感知的通信调度，在异构网络环境下容易形成通信瓶颈，限制整体训练吞吐。因此，在异构 GPU 集群上高效训练 MLLM，仍需在 ZeRO 与 Megatron 等现有框架基础上，引入对设备能力与模型结构的感知与自适应优化。

1.3 研究内容与贡献

1.3.1 MMoH: 多模态大语言模型在异构 GPU 集群上的高效混合并行

本课题结合现有并行策略的不足，考虑实际 GPU 集群的异构场景，针对实际 GPU 集群中存在的存储、算力和网络的异构情况，旨在设计并实现一个高效的、细粒度的混合自动并行编译优化技术，提供一个面向异构复杂环境下多模态大语言模型并行训练的自适应编译优化方案，有效解决异构 GPU 集群下计算效率低、存储和通信开销瓶颈等问题，提升多模态大语言模型并行训练的效率。

针对异构 GPU 设备环境下的存储异构导致的存储空间利用不均衡的问题、通信异构带来的通信瓶颈问题、算力异构带来的 GPU 利用率不高的问题、模型异构带来的算力分配问题、冻结对于显存和计算需求的变化问题，本课题提出一种综合的、面向异构设备和异构模型的多模态大语言模型计算图的层级划分方法，通过为不同属性的计算设备分配不同大小的部分模型，实现计算资源、存储空间、通信带宽的充分利用，争取最大限度提高训练速度。

MLLM 训练与常见的大语言模型的训练过程不同，通常需要分为不同的阶段进行训练，在不同的阶段中对不同模块进行冻结，冻结部分的反向计算量远小于未冻结部分，进一步加剧阶段间执行时间的不均衡，使得气泡率在异构流水线中居高不下，而且可能存在冗余的计算和通信。

本课题针对上述问题，提出一种面向多模态大语言模型的提前终止流水线调度策略。该策略可以感知模型在不同阶段的冻结情况，根据各阶段的冻结情况，调整模型的划分策略；同时结合冻结特性，检测冗余的计算和通信操作并且直接丢弃，提高多卡流水线的有效算力利用率与训练吞吐。

1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的异构梯度检查点

激活检查点（梯度检查点）通过在前向时丢弃部分中间激活、在反向时按需重算，以计算换显存，是支撑大模型与 MLLM 在有限显存下训练的重要手段。然

而，现有梯度检查点策略多采用全局统一的配置（如对全部层或按固定间隔插入检查点），未考虑异构集群中设备间显存与算力的差异：显存紧张、算力相对充裕的设备适合更激进的检查点以释放显存；显存宽裕、算力紧张或通信繁忙的设备则不宜过度重算，否则会加重计算或通信瓶颈。此外，MLLM 各子模块（Encoder、LLM、Projector、Generator）的层数、每层激活体积与计算量各不相同，在同一设备上对不同模块采用相同检查点策略也难以达到最优。

本课题针对上述不足，提出一种面向异构设备与异构模型的梯度检查点优化方法。该方法在给定层级划分与设备映射的基础上，以各设备的显存上限、算力及通信负载为约束，为不同设备上的不同模型阶段分别搜索或配置差异化的检查点策略（如检查点插入的粒度与密度），在满足显存不溢出的前提下，最小化因重计算带来的额外时间开销，并兼顾流水线各阶段的负载均衡。通过将梯度检查点与前述的复合粒度划分、流水线调度相结合，实现计算、存储与通信在异构环境下的联合优化，进一步提升 MLLM 在异构 GPU 集群上的训练效率。

1.4 论文组织结构

本论文共分为五章，各章内容安排如下：

第一章为绪论，介绍了面向异构 GPU 集群的多模态大语言模型并行训练的研究背景与意义，分析了多模态大语言模型面临的挑战与异构 GPU 集群并行训练策略的现状，并概括了本文的主要研究内容与贡献。

第二章为相关工作，从多模态大语言模型架构与训练、并行与分布式训练基础、异构 GPU 集群并行训练策略以及梯度检查点优化等方面对国内外相关研究进行综述。

第三章、第四章分别介绍本文提出的面向异构计算设备与模型的计算图划分方法、流水线调度方法、梯度检查点优化方法及实验验证。

第五章为总结与展望，对全文工作进行总结，并指出未来可能的研究方向。

第二章 相关工作

本章从多模态大语言模型架构与训练、并行与分布式训练基础、异构 GPU 集群并行训练策略以及梯度检查点优化等方面对国内外相关研究进行综述。

基于 Transformer 的大语言模型在自然语言处理 [??]、图像识别 [??]、多模态理解与生成 [??] 等众多领域取得了显著进展；多模态大语言模型（MLLMs）在此基础上融合视觉、语言与生成模块，成为当前研究的热点之一。然而，缩放律 Scaling Law[?] 表明模型性能随参数规模与数据规模提升而持续改善，工业界与学术界因此不断推动模型参数量与训练数据量的增长，单设备在显存与算力上已无法容纳完整模型与训练流程 [?]，并行与分布式训练成为训练大模型与 MLLM 的必由之路。目前，以数据并行、张量并行、流水线并行及零冗余优化器（ZeRO）等为代表的并行策略已被广泛应用于千亿级参数模型的训练 [???]，但上述框架在设计时多假设同构 GPU 集群与统一的设备拓扑，对实际环境中普遍存在的设备与网络异构考虑不足。

在实际部署中，高校、研究所与企业的集群往往由多代、多型号 GPU 混合组成，存在显存异构、算力异构与通信异构 [??]。在此类异构环境下直接使用传统并行策略，易出现负载不均、通信瓶颈与“木桶效应”，导致整体训练效率与资源利用率显著下降；而 MLLM 的模块异构（Encoder、LLM、Projector、Generator 结构各异）与分阶段冻结策略，进一步增加了在异构集群上高效并行训练的难度。

因此，下文首先介绍多模态大语言模型的架构、训练数据与训练目标；随后综述常见并行策略（数据并行、张量并行、流水线并行等）与 ZeRO 系列优化；接着讨论异构 GPU 集群下的并行训练研究现状及其局限；最后介绍梯度检查点等显存优化技术，并说明现有工作为何难以直接支撑面向异构集群的 MLLM 高效训练，从而引出本文的研究内容。

2.1 多模态大语言模型

2.1.1 多模态大语言模型架构

多模态大语言模型（MLLMs）将视觉、语言与（可选）生成能力统一到以大规模语言模型为核心的架构中，其典型组成如图??所示，主要包括以下几类模块。

Modality Encoder 负责将图像、视频、音频等非文本输入编码为高维特征表示，以便与语言模型的词嵌入空间对接。实践中多采用在大规模图文或视频数据上预训练好的视觉模型作为编码器：CLIP[?] 通过对比学习实现开放词汇的视觉表征，被 LLaVA[?]、MiniGPT-4[?] 等广泛采用；ViT[?] 及规模化变体 [?] 则将图像

切分为 patch 后经 Transformer 编码，在 Flamingo[?] 等工作中用作视觉 backbone。编码器输出通常为序列形式的特征，再经后续模块映射到 LLM 空间。

Input Projector 将编码器输出的多模态特征映射到与 LLM 词嵌入维度与语义空间对齐的表示，使 LLM 能够将视觉等信息与文本统一处理。常见实现包括线性层、轻量 Transformer 层，以及 BLIP-2[?] 提出的 Q-Former（通过可学习的 query 从编码器特征中抽取与任务相关的表示）。该模块参数量远小于 LLM，常与编码器一起在预训练或对齐阶段更新。

LLM Backbone 即大规模语言模型主体，承担主要的语言理解与生成计算，参数量通常占整个 MLLM 的 80%–90%[?]。基座模型多选用已在海量文本上预训练好的 Transformer 模型，如 GPT 系列[?]、LLaMA[?]、T5[?] 等；在资源受限或强调高效微调时，常对 LLM 进行冻结或仅微调部分层。

Output Projector 与 *Modality Generator* 用于多模态生成任务（如图生图、文生图、文生视频等）：前者将 LLM 的输出映射到生成器可接受的表示空间，后者根据该表示生成图像、视频或音频。生成器多采用扩散模型，如 Stable Diffusion[?] 中基于 U-Net[?] 的潜在扩散结构；Next-GPT[?]、DreamLLM[?] 等工作在此基础上实现了任意模态到任意模态的生成与编辑。

BLIP-2[?] 率先系统性地采用“冻结 Encoder + 冻结 LLM + 仅训练轻量 Projector”的范式，在显著降低训练成本的同时达到与全参数微调可比的效果，后续多数 MLLM 均沿用或发展此类设计。这种模块异构（编码器、投影层、LLM、生成器在结构、参数量与计算特性上差异显著）与分阶段冻结策略，使得在异构 GPU 集群上为各模块分配合适的算力与显存、并设计均衡的并行与调度策略变得尤为困难。

2.1.2 多模态大语言模型训练数据

缩放律 Scaling Law[?] 表明，模型性能随训练数据规模与模型参数规模的提升而持续改善，因此工业界与学术界在预训练与微调阶段均倾向于使用尽可能大的数据与模型。大语言模型通常在海量文本语料（如 Common Crawl、书籍与代码）上预训练，规模可达数万亿 token；多模态大语言模型在此基础上还需大规模图文对、视频—文本对等多模态数据，用于模态对齐、指令微调与多模态生成等阶段，数据量与存储需求进一步放大。

表??列举了若干代表性数据集的规模与用途。图像方面，Open Images[?] 包含约 900 万张图像及框注，存储需求约 18TB；JFT、LAION 等内部或开放图文对数据集规模可达数亿至数十亿样本，用于视觉—语言预训练。视频方面，YouTube-8M[?] 等基准包含数百万视频与标签，用于视频理解与生成的数据集往往需要 PB 级存

储与高吞吐的数据管线。文本方面，Common Crawl、The Pile 等为 LLM 预训练提供数万亿 token 的语料。在单机显存与存储均无法容纳完整模型与全量数据的条件下，并行训练（数据并行、模型并行及其组合）成为必然选择；数据规模与模型规模的持续增长，也对并行策略的 I/O、通信效率与异构适应性提出了更高要求[?]。

表 2.1 多模态大语言模型训练中常用的部分数据集规模示意

类型	规模量级	存储量级	典型用途
图像	数百万 – 数十亿	TB–PB	图文预训练、对齐（如 Open Images[?]）
视频	数百万 – 数亿	百 TB–PB	视频理解、生成（如 YouTube-8M[?]）
文本 / 语料	数百亿 – 数万亿 token	PB 级	LLM 预训练、指令数据

2.1.3 多模态大语言模型训练目标

多模态大语言模型的训练通常采用多阶段、分模块的策略，在不同阶段设定不同的训练目标与可训练参数集合，以兼顾效果与成本[?]。典型阶段包括：（1）模态对齐（或预训练）：在大规模图文对或视频—文本对上训练 Input Projector（及可选的部分 Encoder/LLM），使视觉等特征与 LLM 语义空间对齐；（2）指令微调（Instruction Tuning）：在高质量指令—回答数据上微调，以提升遵循指令与对话能力，此时常冻结 Encoder、仅微调 Projector 与 LLM 部分层；（3）多模态生成微调（若需文生图等）：在生成数据上训练 Output Projector 与 Modality Generator，LLM 可冻结或低秩微调。各阶段对“谁冻结、谁更新”的设定不同，从而在同一模型上形成不同的计算图与显存 / 算力需求分布。

冻结会显著改变各模块的反向计算量与显存占用：冻结参数不参与梯度计算与更新，优化器状态（如 Adam 的动量与方差）仅需为可训练参数维护，因此冻结比例越高，单步显存与计算量通常越低，但可调表达能力也越受限[?]。在流水线并行或层级划分下，若某流水级被冻结，其对应设备上的反向计算远少于未冻结阶段，容易造成流水级间负载不均与流水线气泡；在异构 GPU 集群下，这种因训练阶段与冻结策略带来的“动态模型异构”与设备异构叠加，进一步增加了并行策略设计调优的难度。

图??概括了上述多阶段训练中常见的模块冻结与更新关系，便于理解不同阶段对算力与显存的需求差异。



图 2.1 MLLM 多阶段训练中典型模块冻结与更新关系示意

2.2 并行与分布式训练基础

2.2.1 数据并行与模型并行

数据并行（DP）。 数据并行将训练数据在各计算设备间划分，每个设备持有一份完整模型副本，本地前向与反向后通过梯度 All-Reduce 等通信同步参数 [? ? ?]。如图??所示，各设备独立计算所分配 mini-batch 的梯度，再在迭代末聚合梯度并更新一致参数。DP 实现简单、扩展性好，是 PyTorch DDP[?]、Horovod[?] 等框架的默认方式；Parallax[?] 针对稀疏与异构场景对参数服务器与 All-Reduce 做了灵活选择。其局限在于单卡需容纳完整模型与优化器状态，无法单独支撑超大参数规模，常与张量 / 流水线并行或 ZeRO 组合使用。

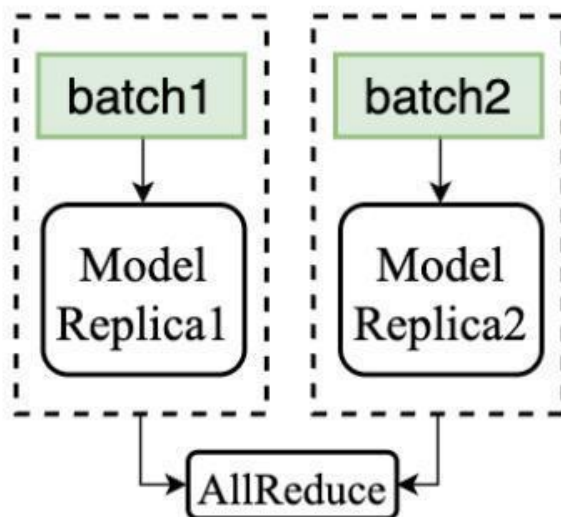


图 2.2 数据并行示意

张量并行（TP）与序列并行（SP）。 张量并行在层内划分权重与计算，使多设备协同完成单层的前向与反向，组合后等价于完整层 [? ?]。如图??所示，每设备仅持有部分参数与激活，层间通过 All-Gather/Reduce-Scatter 等通信传递激活与

梯度。Megatron-LM[?] 针对 Transformer 提出按行 / 列交替划分 FFN 与注意力权重，在减少同步次数的同时控制通信量；GShard[?] 在 MoE 中结合张量并行与专家划分。TP 通信密集且对带宽敏感，实践中多限于单节点或高带宽拓扑内，并与流水线、数据并行组合 [?]。序列并行（SP）针对长序列场景，将输入沿序列维划分到多卡，每卡仅处理一段子序列，通过通信在注意力等层交换信息以保持语义一致 [?]。DeepSpeed-Ulysses[?] 将序列均匀分片，在注意力处采用两阶段 All-to-All 重分布，使通信量随序列长度与设备数成比例缩放，支持百万级 token 训练；Megatron 中的实现常与张量并行共用进程组，与 TP、PP、DP 组合构成多维混合并行 [?]。

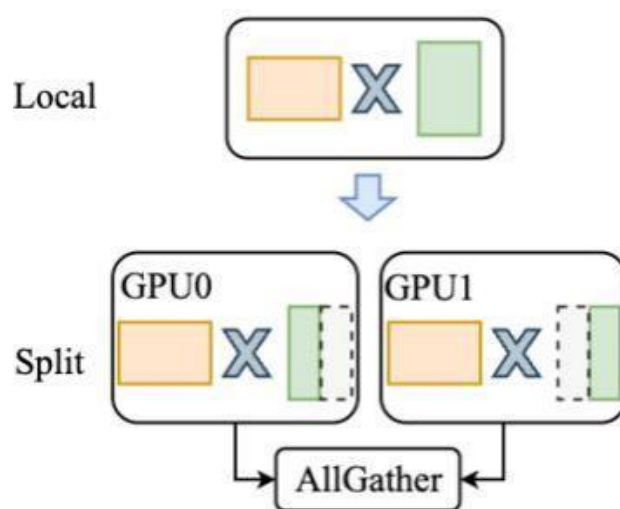


图 2.3 张量并行示意

流水线并行（PP）。 流水线并行在层间划分模型，每设备持有若干连续层构成一个 stage，多个 micro-batch 依次流过各 stage 以重叠计算、降低空闲（bubble）[?]。如图??所示，GPipe[?] 通过 micro-batch 与多阶段调度减少 bubble；PipeDream[?] 提出 1F1B（1 Forward 1 Backward）等异步调度进一步压缩空闲；TeraPipe[?] 在序列内做 token 级流水线，实现更细粒度并行。近年来，ZeroBubble[?] 在同步语义下将反向拆分为“对输入的梯度”与“对参数的梯度”两阶段，结合 1F1B1W 等调度与优化步同步旁路，实现了近似零 bubble，在相同显存下相较 1F1B 提升吞吐约 23%–31%；DualPipe[?] 采用双向 micro-batch 流动与前后向重叠，在 DeepSeek 等超大规模训练中用于降低 bubble 与通信压力。PP 通信量相对较小，适合跨节点扩展，常与节点内 TP、全局 DP 组合 [?]。

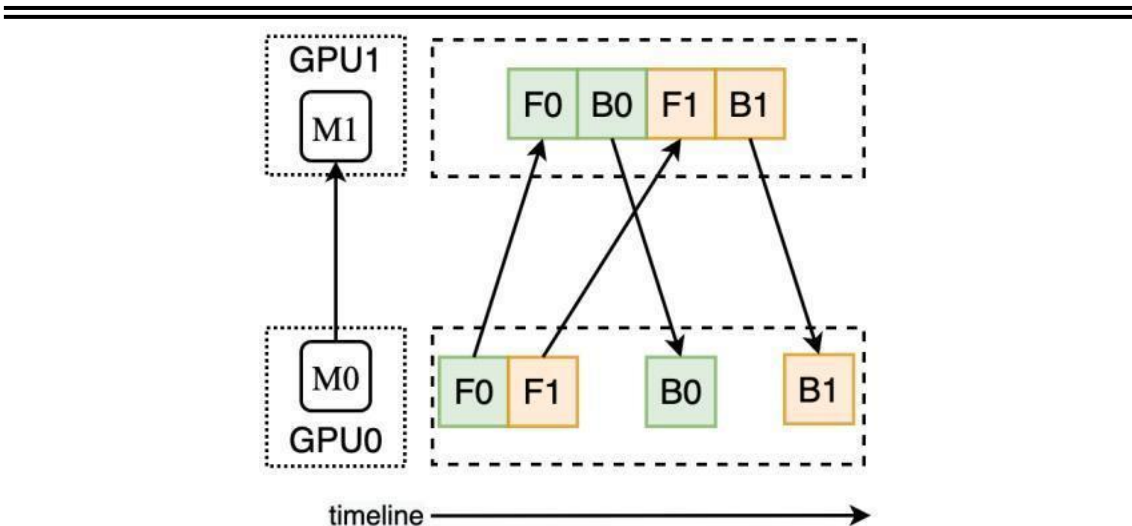


图 2.4 流水线并行示意

上下文并行（CP）与专家并行（EP）。上下文并行（CP）与序列并行在长序列场景下思路相近：单卡显存难以容纳全长序列的激活时，将输入沿序列维划分到多卡，在注意力等层通过通信交换信息以保持语义一致，Megatron 中的实现常与张量并行共用进程组，称为 Context Parallelism（CP）[?]。CP 在 8K+ token 场景下显存收益显著，但会引入额外通信与拓扑约束，需与 Flash Attention、激活检查点等配合使用。专家并行（EP）针对混合专家模型（MoE）：不同 token 由不同子网络（专家）处理，EP 将完整专家分布到不同设备，每设备仅持有部分专家权重，前向时根据路由将激活发送至对应专家所在设备，再汇总结果[?]。与张量并行（每设备持所有专家的部分权重）不同，EP 下每设备只存部分专家的全量权重，通信模式为 All-to-All 的 token 重分布，适合专家数多、单专家规模适中的 MoE。GShard[?] 将 MoE 与张量并行结合并做自动分片；DualPipe[?] 等也在流水线中结合专家并行以支撑超大规模 MoE 训练。EP 的主要挑战是专家负载不均与路由带来的通信开销，需高带宽互联与负载均衡设计。图??概括了 SP/CP 与 EP 的划分维度与数据流关系。

图 2.5 序列并行（SP）/ 上下文并行（CP）与专家并行（EP）的划分维度示意

2.2.2 零冗余优化器（ZeRO）

ZeRO[? ?] 在数据并行基础上，将优化器状态、梯度与参数分片存于各卡，前向与反向时按需聚合，在保持 DP 通信量的前提下显著降低单卡显存，使千亿级训练可行。如图??所示，ZeRO 三阶段分别分片优化器状态、梯度与参数；ZeRO-Infinity[?] 进一步将部分状态卸载至 CPU/NVMe 以突破 GPU 显存墙。

ZeRO++[?] 通过量化与通信重算减少跨节点通信量；MiCS[?]、AMSP[?] 等在云环境与统一划分空间上做了扩展。ZeRO 与 TP、PP、SP 等组合构成当前大模型与 MLLM 训练的主流基础设施。

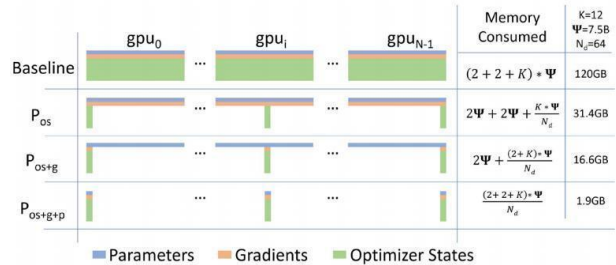


图 2.6 零冗余优化器 ZeRO 示意

2.3 异构 GPU 集群并行训练策略

实际部署中，高校、企业与云平台往往无法配备全同构的 GPU 集群：设备多代共存（如 A100 与 V100、H100 混用）、显存容量从数十 GB 到 80 GB 不等、算力与互联带宽差异显著。在此类异构环境下，若直接套用为同构集群设计的数据并行、张量并行与流水线并行，会暴露出三方面问题。存储异构方面，传统策略多假设各卡显存一致，按均匀方式划分模型与激活，导致大显存设备利用率不足而小显存设备易发生 OOM，制约扩展性。算力异构方面，同步式并行要求各设备步调一致，最慢设备成为瓶颈，产生“木桶效应”，整体算力利用率与训练吞吐下降。网络异构方面，集群常存在多级互联（节点内 NVLink、节点间 InfiniBand 或以太），不同链路带宽与延迟差异大，跨节点通信易成为瓶颈。因此，面向异构 GPU 集群的并行训练策略成为近年来的研究热点，下文对代表性工作及其局限进行综述。

HetPipe[?] 较早将流水线并行与数据并行结合以利用异构集群。其将多个 GPU 组织为虚拟工作节点（VW），在 VW 内采用流水线模型并行（PMP）处理小批量，在 VW 间采用数据并行提升可扩展性。通过将“弱”设备与“强”设备搭配成 VW，在一定程度上缓解了单机算力不足的问题。然而，HetPipe 未显式建模 VW 内部的异构设备排列与通信拓扑，对通信与显存开销的估计较为直观，且未考虑模型异构（如 MLLM 中 Encoder、Projector、LLM 模块结构各异、冻结策略不同），在当今多模态大语言模型与分阶段训练场景下适用性有限。

Whale[?] 针对显存与算力异构，提出基于硬件感知的负载均衡方法。通过刻画各 GPU 的存储与算力差异，Whale 按比例对各设备分配不同大小的批次与模型分片，在数据并行与张量并行下实现计算与显存负载的均衡。其思路对“大卡多

算、小卡少算”的配置有参考价值。但 Whale 未考虑网络异构，且存储 / 算力均衡策略相对直观，划分与编译粒度停留在子图级别，未与 ZeRO 等显存优化结合，在超大规模与多维度异构并存时仍有优化空间。

AMP[?] 在异构环境下为大模型自动寻找并行策略。AMP 同时考虑模型异构（不同层结构、参数量与计算量）与设备异构（GPU 显存、算力、网络），对层间差异进行抽象与建模，对集群内存储与网络异构进行近似量化，在此基础上基于 3D 并行（DP+TP+PP）设计开销模型（Cost Model），并用动态规划求解近似最优的并行配置。实验表明其在异构场景下相较基线有明显提升。不足在于：划分粒度为全局粗粒度，开销模型对通信与显存估计的准确性有限，且未显式约束显存上界，实际部署中易出现 OOM，仍需与显存预算和梯度检查点等联合考虑。

近年来，面向大模型在云上与异构集群中训练的资源调度与配置规划工作受到关注。Espresso[?] 是一种面向成本效益的异构 GPU 资源供给与阶段放置框架，面向大规模模型训练场景。Espresso 集成四大模块：Profiler 分析不同层在异构 GPU 上的迭代时间与显存占用；GPU Allocator 负责异构 GPU 的分配；Stage Placer 确定流水线等并行策略下的阶段放置；Performance Estimator 预测给定配置下的训练性能。用户可指定模型、数据集、训练轮数与 SLO（服务等级目标），由系统自动给出资源分配与 stage 放置方案，从而在异构环境中兼顾成本与效率。Espresso 为异构集群上的大模型训练提供了从分析到配置的一体化思路，其与 ZeRO、流水线气泡优化及 MLLM 模块异构的结合仍可进一步探索。

Zorse[?] 则直接针对异构 GPU 集群上的 LLM 训练效率优化。其指出，在异构集群上高效训练需要：（1）将流水线并行与数据并行以通信与显存高效的方式结合；（2）支持更灵活的配置，包括将 GPU 灵活划分为非对称的流水线阶段、在同一数据并行组内容纳异构 GPU。Zorse 实现了上述能力的统一，并配备自动规划器，根据工作负载自动生成训练配置。实验表明在异构场景下相较现有系统有显著提升。Zorse 与 Espresso 等工作的出现，反映了社区对异构环境下大模型训练系统化方法的需求，二者在 MLLM 多模块、分阶段冻结与多模态数据管线等场景下的适配仍是开放问题。

此外，与 ZeRO 系列相关的工作在异构场景下也存在局限。ZeRO[?] 的梯度 / 参数 All-Gather 等通信在跨节点异构网络中易成为瓶颈；MiCS[?] 对模型规模与异构网络下的高带宽链路利用仍有不足；ZeRO++[?] 的量化与通信重算在带来收益的同时可能引入精度与通用性代价；AMSP[?] 等侧重跨节点通信削减，对集群内存储、算力与带宽的细粒度异构利用仍有限。综合来看，现有异构并行与 ZeRO 类方法多针对通用 LLM 或同构假设下的扩展，对多模态大语言模型的模块异构、分阶段冻结与显存—算力—通信三重异构的联合优化仍不足，为本文面向

异构集群的 MLLM 高效训练与调度研究留下了空间。

2.4 梯度检查点优化

大规模模型与长序列训练中，前向传播产生的中间激活（activation）需在反向传播时用于梯度计算，若全部保存则显存占用随层数与序列长度线性甚至平方增长，成为训练瓶颈。梯度检查点（Gradient Checkpointing，又称激活检查点或激活重计算）通过“以计算换显存”：前向时仅保存部分激活或仅保存检查点处的输入，反向时对未保存的区段重新执行前向以得到所需激活，从而在显存与计算之间进行权衡 [?]。根据检查点放置的粒度和策略，可大致分为全部重计算、选择重计算与细粒度重计算三类。

全部重计算。 全部重计算（full recomputation）策略在前向阶段不保存任何中间激活，仅在若干“检查点”层保存该层的输入（或仅保存网络输入），反向传播需要某段中间激活时，从最近的检查点重新执行前向至该位置再计算梯度。极端情况下仅保存网络输入，则显存占用可降至与深度无关的常数级别，但反向阶段需对整网做一次完整前向重算，计算开销约增加一倍 [?]。Chen 等 [?] 针对链式计算图证明了在仅增加一次额外前向的前提下，通过动态规划可求得检查点的最优放置，使显存从 $O(n)$ 降为 $O(\sqrt{n})$ （ n 为层数），并在千层残差网络等实验中实现了显存的大幅下降（如 48 GB $\rightarrow$ 7 GB）与约 30% 的训练时间增加。全部重计算实现简单、显存收益最大，适合显存极度紧张、算力相对充裕的场景，但计算冗余较高。

选择重计算。 选择重计算（selective recomputation）只将部分层或模块选为检查点保存激活，其余层在反向时重算。通过合理选择检查点位置，可在显存与计算之间取得比“全存”或“全不存”更优的折中。Chen 等 [?] 的 $O(\sqrt{n})$ 算法即为一种选择重计算的最优策略（在给定计算预算下最小化显存）。实践中，常按层或块（如每 k 层存一次、或每个 Transformer block 的输入存一次）设置检查点，在 Transformer 中若对每层输入做检查点，激活显存可显著下降，但重算开销约增加 30% 量级。选择重计算被 PyTorch (`torch.utils.checkpoint`)、Megatron-LM、NeMo 等框架广泛支持，用户可通过指定哪些子模块参与 checkpoint 来调节显存—计算权衡。

细粒度重计算。 细粒度重计算（fine-grained recomputation）在算子或更小粒度上决定哪些激活保存、哪些重算，从而更精细地利用不同算子的计算—显存特性。例如，Transformer 中自注意力的激活随序列长度平方增长，而线性层等算

子的重算成本与显存占用比例与注意力不同，若仅在注意力块内做选择性重算、或对 LayerNorm/Dropout 等轻量层不存而重算，可进一步降低显存同时控制重算量。Korthikanti 等 [?] 针对大尺度 Transformer 提出序列并行与选择性激活重计算 (selective activation recomputation)：在张量并行基础上，将序列维划分到多设备，并仅对必要的激活做重算，其余通过通信与局部计算避免冗余。在万亿参数规模语言模型上的实验表明，该方法可将激活显存降低约 5 倍，并将重算带来的时间开销减少 90% 以上，例如在 530B 参数 GPT 风格模型上相比传统重算获得约 29% 的训练加速 [?]。细粒度策略可结合模型结构（如 MLLM 中 Encoder、Projector、LLM 各模块的算子组成）与异构设备的显存、算力差异，在不同设备或不同 stage 上采用不同的检查点粒度，是面向异构集群与 MLLM 的潜在优化方向。

在 Megatron-LM 与 NeMo 等框架中，激活重计算被实现为可配置的多级粒度。全层重计算 (full) 以整个 Transformer 层为粒度：检查点保存每层输入，反向时重算整层前向，显存下降显著但单层计算约增加 30%。框架进一步提供两种全层模式：*block* 模式通过参数指定每个流水线 stage 内参与重计算的层数，可仅对部分层做检查点以在显存与计算间折中；*uniform* 模式以连续多层的“块”为重计算单位，块内仅存块首输入，适用于层数很多或层间激活相对较小的模型，可节省激活显存但会增加重算缓冲。选择性重计算 (selective) 则仅在子模块级别重算：例如仅对自注意力块做检查点与重算（保存注意力块输入，重算 softmax、dropout、QKV 等中间激活），而 MLP 等不重算。自注意力激活随序列长度平方增长、占显存大头，但其重算成本相对线性投影层较低，因此选择性重算能以较小计算代价获得较高显存收益；用户还可指定 `core_attn`、`mlp`、`moe` 等模块是否参与重算。在使用 Flash Attention 时，框架通常强制对注意力做选择性重算以配合其显存布局。上述多级粒度（全层 *block/uniform*、子模块 *selective*）为实际训练中按显存预算与算力调节检查点策略提供了细粒度控制，与 Korthikanti 等 [?] 的序列并行与选择性激活重算相结合，已广泛应用于大规模 Transformer 与 LLM 训练。然而，现有检查点策略多面向同构设备设计：重计算层数、块大小、参与重算的子模块等均需用户或策略脚本手动指定，且通常假设各设备显存与算力一致，无法根据异构集群中不同设备的显存余量与算力差异自动调节（例如在显存紧张设备上加大重算比例、在算力瓶颈设备上减少重算）。

将梯度检查点与流水线并行、stage 划分及异构分配结合，可进一步缓解显存瓶颈并提升整体吞吐：例如在流水线各 stage 内独立设置检查点策略，或在显存紧张的设备上采用更激进的重计算、在算力紧张处适当多存以减少重算，但上述调节目前仍依赖人工或离线调参。Colossal-Auto[?] 等将自动并行策略搜索与激活检查点统一自动化，为大规模模型与异构环境下的联合优化提供了参考；在异构

GPU 集群上，针对 MLLM 的模块异构与分阶段冻结，设计能自动适应设备异构的细粒度检查点策略仍是开放问题，也是本文后续工作方向之一。

致 谢

作者在学期间取得的学术成果

发表的学术论文

- [1] Yang Y, Ren T L, Zhang L T, et al. Miniature microphone with silicon- based ferroelectric thin films. *Integrated Ferroelectrics*, 2003, 52:229-235. (SCI 收录, 检索号:758FZ.)
- [2] 杨轶, 张宁欣, 任天令, 等. 硅基铁电微声学器件中薄膜残余应力的研究. *中国机械工程*, 2005, 16(14):1289-1291. (EI 收录, 检索号:0534931 2907.)
- [3] 杨轶, 张宁欣, 任天令, 等. 集成铁电器件中的关键工艺研究. *仪器仪表学报*, 2003, 24(S4):192-193. (EI 源刊.)
- [4] Yang Y, Ren T L, Zhu Y P, et al. PMUTs for handwriting recognition. In press. (已被 *Integrated Ferroelectrics* 录用. SCI 源刊.)
- [5] Wu X M, Yang Y, Cai J, et al. Measurements of ferroelectric MEMS microphones. *Integrated Ferroelectrics*, 2005, 69:417-429. (SCI 收录, 检索号:896KM.)
- [6] 贾泽, 杨轶, 陈兢, 等. 用于压电和电容微麦克风的体硅腐蚀相关研究. *压电与声光*, 2006, 28(1):117-119. (EI 收录, 检索号:06129773469.)
- [7] 伍晓明, 杨轶, 张宁欣, 等. 基于 MEMS 技术的集成铁电硅微麦克风. *中国集成电路*, 2003, 53:59-61.

研究成果

- [1] 任天令, 杨轶, 朱一平, 等. 硅基铁电微声学传感器畴极化区域控制和电极连接的方法: 中国, CN1602118A. (中国专利公开号.)
- [2] Ren T L, Yang Y, Zhu Y P, et al. Piezoelectric micro acoustic sensor based on ferroelectric materials: USA, No.11/215, 102. (美国发明专利申请号.)

附录 A 模板提供的希腊字母命令列表

大写希腊字母:

Γ \Gamma	Λ \Lambda	Σ \Sigma	Ψ \Psi
Δ \Delta	Ξ \Xi	Υ \Upsilon	Ω \Omega
Θ \Theta	Π \Pi	Φ \Phi	
Γ \varGamma	Λ \varLambda	Σ \varSigma	Ψ \varPsi
Δ \varDelta	Ξ \varXi	Υ \varUpsilon	Ω \varOmega
Θ \varTheta	Π \varPi	Φ \varPhi	

小写希腊字母:

α \alpha	θ \theta	o o	τ \tau
β \beta	ϑ \vartheta	π \pi	υ \upsilon
γ \gamma	ι \iota	ϖ \varpi	ϕ \phi
δ \delta	κ \kappa	ρ \rho	φ \varphi
ϵ \epsilon	λ \lambda	ϱ \varrho	χ \chi
ε \varepsilon	μ \mu	σ \sigma	ψ \psi
ζ \zeta	ν \nu	ς \varsigma	ω \omega
η \eta	ξ \xi	κ \varkappa	$\digamma$ \digamma
α \upalpha	θ \uptheta	o \mathrm{o}	τ \uptau
β \upbeta	ϑ \upvartheta	π \uppi	υ \upupsilon
γ \upgamma	ι \upiota	ϖ \upvarpi	ϕ \upphi
δ \updelta	κ \upkappa	ρ \uprho	φ \upvarphi
ϵ \upepsilon	λ \uplambda	ϱ \upvarrho	χ \upchi
ε \upvarepsilon	μ \upmu	σ \upsigma	ψ \uppsi
ζ \upzeta	ν \upnu	ς \upvarsigma	ω \upomega
η \upeta	ξ \upxi		

希腊字母属于数学符号类别, 请用\bm命令加粗, 其余向量、矩阵可用\mathbf。